SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 40%

QUESTIONS: 03 **Total Marks:25**

Annexure A

Coding Challenge

Code of Conduct:

I Abinesh H (192424387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Total*
Weightage	*30%*	*20%*	*15%*	*20%*	*15%*	
Q1						
Q2						
Q3						

Signature of the student: Abinesh H ***Total Marks :***

Annexure A

Short Answer Test

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

2. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

3. Write a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Naïve Bayes Classifier

```
import pandas as pd
from collections import Counter

# Dataset
data = pd.DataFrame({
   'Outlook':['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny','Sunny','Rain'],
   'Temperature':['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','Cool','Mild'],
   'Play':['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes']
})

test = {'Outlook':'Sunny', 'Temperature':'Cool'}

# Naive Bayes function
def naive_bayes(data, test, smooth=False):
   classes = data['Play'].unique()
   result = {}

   for c in classes:
      subset = data[data.Play == c]
      prior = len(subset) / len(data)
      prob = prior

      for feature, value in test.items():
         count = ((subset[feature] == value).sum())

         if smooth:
            prob *= (count + 1) / (len(subset) + data[feature].nunique())
         else:
            prob *= count / len(subset)

      result[c] = prob

   return max(result, key=result.get), result

# Without smoothing
pred1, p1 = naive_bayes(data, test, False)

# With Laplace smoothing
pred2, p2 = naive_bayes(data, test, True)

print("Prior Probabilities:")
print(data.Play.value_counts(normalize=True))

print("\nConditional Probabilities:")
for c in data.Play.unique():
   print(c)
   s = data[data.Play == c]
   for f in ['Outlook','Temperature']:
```

```
        print(f, s[f].value_counts(normalize=True).to_dict())

print("\nWithout Laplace Smoothing:")
print(p1)
print("Predicted Class:", pred1)

print("\nWith Laplace Smoothing:")
print(p2)
print("Predicted Class:", pred2)

# Accuracy on training data
for smooth in [False, True]:
    correct = 0
    for _, row in data.iterrows():
        test_row = {'Outlook':row.Outlook, 'Temperature':row.Temperature}
        pred, _ = naive_bayes(data.drop(_, errors='ignore'), test_row, smooth)
        correct += pred == row.Play

    print("Accuracy with smoothing =", smooth, ":", correct/len(data))
```

Output:

```
Prior Probabilities:
Play
Yes    0.6
No     0.4
Name: proportion, dtype: float64

Conditional Probabilities:
No
Outlook {'Sunny': 0.75, 'Rain': 0.25}
Temperature {'Hot': 0.5, 'Cool': 0.25, 'Mild': 0.25}
Yes
Outlook {'Rain': 0.5, 'Overcast': 0.3333333333333333, 'Sunny': 0.16666666666666666}
Temperature {'Cool': 0.5, 'Mild': 0.3333333333333333, 'Hot': 0.16666666666666666}

Without Laplace Smoothing:
{'No': np.float64(0.07500000000000001), 'Yes': np.float64(0.049999999999999996)}
Predicted Class: No

With Laplace Smoothing:
{'No': np.float64(0.06530612244897958), 'Yes': np.float64(0.059259259259259255)}
Predicted Class: No
Accuracy with smoothing = False : 0.6
Accuracy with smoothing = True : 0.6
```

2. Genetic Algorithm

```
import random

# Fitness function: maximize f(x) = x^2
def fitness(x):
    return x ** 2

# Parameters
POP_SIZE = 10
GENERATIONS = 20
MUTATION_RATE = 0.1

# Initial population
population = [random.randint(-10, 10) for _ in range(POP_SIZE)]

for gen in range(GENERATIONS):
    scores = [fitness(x) for x in population]

    # Selection
    parents = sorted(population, key=fitness, reverse=True)[:4]

    # Crossover
    new_population = parents.copy()

    while len(new_population) < POP_SIZE:
        p1, p2 = random.sample(parents, 2)
        child = (p1 + p2) // 2

        # Mutation
        if random.random() < MUTATION_RATE:
            child += random.choice([-1, 1])

        child = max(-10, min(10, child))
        new_population.append(child)

    population = new_population

    best = max(population, key=fitness)
    print(f"Generation {gen+1}: Best = {best}, Fitness = {fitness(best)}")

print("\nBest Solution:", best)
print("Best Fitness:", fitness(best))
```

Output:

```
Generation 1: Best = -9, Fitness = 81
Generation 2: Best = -10, Fitness = 100
Generation 3: Best = -10, Fitness = 100
Generation 4: Best = -10, Fitness = 100
Generation 5: Best = -10, Fitness = 100
Generation 6: Best = -10, Fitness = 100
Generation 7: Best = -10, Fitness = 100
Generation 8: Best = -10, Fitness = 100
Generation 9: Best = -10, Fitness = 100
Generation 10: Best = -10, Fitness = 100
Generation 11: Best = -10, Fitness = 100
Generation 12: Best = -10, Fitness = 100
Generation 13: Best = -10, Fitness = 100
Generation 14: Best = -10, Fitness = 100
Generation 15: Best = -10, Fitness = 100
Generation 16: Best = -10, Fitness = 100
Generation 17: Best = -10, Fitness = 100
Generation 18: Best = -10, Fitness = 100
Generation 19: Best = -10, Fitness = 100
Generation 20: Best = -10, Fitness = 100

Best Solution: -10
Best Fitness: 100
```

3. Support Vector Machine (SVM)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Dataset
X = np.array([
    [1,1], [2,1], [2,2], [3,2],
    [6,6], [7,6], [7,7], [8,7]
])

y = np.array([0,0,0,0,1,1,1,1])

# Train linear SVM
model = SVC(kernel='linear', C=1)
model.fit(X, y)

# Test data
X_test = np.array([[2,2], [3,1], [6,7], [8,8]])
y_test = np.array([0,0,1,1])

# Prediction
```

```
pred = model.predict(X_test)

print("Predicted Labels:", pred)
print("Actual Labels:", y_test)
print("Accuracy:", accuracy_score(y_test, pred))

print("\nSupport Vectors:")
print(model.support_vectors_)

# Plot
plt.scatter(X[:,0], X[:,1], c=y)
plt.scatter(model.support_vectors_[:,0],
            model.support_vectors_[:,1],
            s=150, facecolors='none', edgecolors='black')

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Linear SVM with Support Vectors")
plt.show()
```

Output:

```
Predicted Labels: [0 0 1 1]
Actual Labels: [0 0 1 1]
Accuracy: 1.0

Support Vectors:
[[3. 2.]
 [6. 6.]]
```